

# Production Capstone - Marketplace Data Platform

PROTECTED FINAL BUILD - BEGINNER-FIRST BUILD BOOK

## What this book does NOT do

It does not give you a completed architecture, DAG, dbt model, Spark job or final code. It teaches how to turn each capstone requirement into a decision and evidence gate so you are not left staring at a blank folder.

## Business in plain language

A marketplace receives orders/refunds/seller data from different systems. Analysts need trusted daily Gold tables. Operations also wants a fresh view of delivery problems. Sources can be resent, late, malformed or partially fail. Your platform must preserve source evidence, stop bad data from silently publishing, recover/backfill safely and be explainable.

## First decode the requirements

| Requirement         | Question you must answer                                                              |
|---------------------|---------------------------------------------------------------------------------------|
| paginated API       | How are pages/raw payloads preserved? What if page N fails?                           |
| daily CSV drops     | How do you identify duplicate/redelivered file/business content?                      |
| PostgreSQL source   | How do you extract incrementally and preserve source snapshot metadata?               |
| daily Gold          | What is each Gold table grain and publish quality gate?                               |
| delivery exceptions | Is batch enough? If not, what latency justifies streaming?                            |
| late data/backfill  | What watermark/range reprocess rule prevents corrupting later state?                  |
| schema changes      | What new nullable field can flow through raw safely? What breaking field should stop? |
| partial failure     | Which persisted work can be reused and which tasks rerun?                             |
| security            | Where do secrets live? Which fields are PII? Who can read/write each layer?           |
| 10x scale           | Which bottleneck changes first - ingestion, files, joins, warehouse, orchestration?   |

## Build phases - do not skip the early writing

### Phase 0 - freeze your first attempt folder

Before design changes, create evidence/attempt-01 and note the date/assumptions. This is how you prove learning rather than replacing the project after review.

#### Evidence gate - prove before move-on

- attempt folder exists
- assumptions/unknowns written

#### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

## Phase 1 - source contracts

For API, CSV and database, write grain, business key, timestamps, required/optional fields, duplicate semantics, freshness and schema-change behavior.

### Evidence gate - prove before move-on

- source\_contracts.md covers api/csv/database/grain
- required vs optional clear
- late/redelivery meaning clear

### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

## Phase 2 - output contracts + business controls

Define Gold tables/views and delivery-exception output grain BEFORE architecture. Write count/amount/freshness controls.

### Evidence gate - prove before move-on

- each output has one-row meaning
- business keys defined
- critical reconciliation equations listed

### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

## Phase 3 - architecture decision

Draw the simplest architecture satisfying requirements. Separate raw preservation, trusted transformations, serving, orchestration, monitoring and security boundaries.

### Evidence gate - prove before move-on

- architecture has raw/trusted/orchestration/serving
- failure behavior at each boundary
- Fabric mapped as primary deployment target

### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

## Phase 4 - ingestion/raw landing

Implement or scaffold API pagination, CSV landing and database extract. Preserve raw + batch/source metadata.

### Evidence gate - prove before move-on

- raw/source evidence persisted
- pagination visible
- idempotency/redelivery key explained

### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

## Phase 5 - quality/quarantine/reconciliation

Critical required/duplicate/type/relationship/reconciliation rules must decide publish eligibility.

### Evidence gate - prove before move-on

- quality\_rules.md explicit
- bad data has quarantine/fail path
- critical failure blocks publish

### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

## Phase 6 - trusted relational/lakehouse models

Build dbt/SQL and Spark only where justified. State model grain and joins.

### Evidence gate - prove before move-on

- Gold model uses ref/source or clear lineage
- Spark responsibility justified
- no grain ambiguity

### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.



## Phase 7 - incremental/late/backfill

Define watermarks/merge/business keys and bounded historical repair procedure.

### Evidence gate - prove before move-on

- rerun same batch safe
- late row policy explicit
- bounded backfill documented

### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

## Phase 8 - orchestration/recovery

Airflow/pipeline order must make dependencies and retries safe. Define partial-success recovery.

### Evidence gate - prove before move-on

- DAG has dependency/retry logic
- publish downstream blocked on critical failure
- runbook has recover/rerun/rollback

### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

## Phase 9 - security/CI/observability

Separate secrets, identify PII/access, add tests/CI and freshness/quality/failure/cost monitoring plan.

### Evidence gate - prove before move-on

- no literal secret
- least privilege + PII noted
- CI/test gate
- monitoring covers freshness/quality/failure/cost

### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

## Phase 10 - first end-to-end attempt

Run/check as much of the local implementation as environment permits. Save results and known live-cloud/runtime gaps separately.

### Evidence gate - prove before move-on

- attempt-01 saved
- checker/static tests output saved
- unexecuted cloud/runtime claims labeled honestly

### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

## Phase 11 - open Incident Pack and repair all required incidents

Only now use the incident files. One incident at a time: reproduce -> save failing evidence -> diagnose -> repair -> rerun controls.

### Evidence gate - prove before move-on

- all 8 incidents have before/after evidence
- root cause stated
- repair does not break previous controls

### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

## Phase 12 - publication + defense

Sanitize repo, finish README/runbook/data dictionary/known limitations and complete 5-minute + deep technical defense.

### Evidence gate - prove before move-on

- clean public repo
- reproducible setup
- defense covers trade-offs/recovery/10x/security/cost

### STOP - explain before continuing

Can you point to the exact file/query/log that proves this phase, not merely say you completed it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

## Incident Pack - what each incident is testing

| Incident                     | Underlying competency                            |
|------------------------------|--------------------------------------------------|
| Duplicate source file        | idempotency + source-file/business-key semantics |
| Schema addition              | raw preservation + controlled schema evolution   |
| Bad API credential           | secret hygiene + fail-closed publish behavior    |
| dbt/data test failure        | quality gate blocks Gold                         |
| Spark performance regression | plan/skew diagnosis + justified optimization     |
| Airflow partial failure      | task state + safe rerun/recovery                 |
| Historical gap               | bounded backfill + later-state integrity         |
| Over-permissioned access     | least privilege + PII/write controls             |

## Capstone checker vs real pass

A checker can verify files/keywords/basic structure. It cannot prove your architecture is coherent, your business grain is correct, your incident diagnosis is real, or that you can explain the work. M18 completion requires both machine evidence and human reasoning evidence.

### Evidence gate - prove before move-on

- Critical floor: no hardcoded real credential.
- Critical floor: duplicate/rerun cannot silently double trusted output.
- Critical floor: critical bad data cannot silently publish.
- Critical floor: recovery/backfill path exists.
- Critical floor: access/PII controls are defensible.
- Critical floor: reproducible documented setup.
- Critical floor: you can explain without hidden AI output.